

1. (a) Create a table Student with the following attributes
(Reg_no, Name, DOB, Branch, Year of study Where Reg_no
is primary key) and perform the following:

(i) Implement a NOT NULL constraint on Name
attribute

(ii) Alter the table by adding a new columns 'Address',
Mobileno and EmailId

(iii) Alter the data type from Reg_no Integer to Varchar

(iv) Drop the column Address from the table

(v) Insert data into the table and display all the records

(b) Write a PL/SQL block to display the student details using
procedure.

(a) SQL Commands

Create Table with NOT NULL

```
CREATE TABLE Student (  
    Reg_no INTEGER PRIMARY KEY,  
    Name VARCHAR(50) NOT NULL,  
    DOB DATE,  
    Branch VARCHAR(20),  
    Year_of_study INTEGER  
);
```

(ii) Add new columns

```
ALTER TABLE Student  
ADD (Address VARCHAR(100), Mobileno VARCHAR(15), EmailId VARCHAR(50));
```

(iii) Change datatype of Reg_no

```
ALTER TABLE Student  
MODIFY Reg_no VARCHAR(20);
```

(iv) Drop Address column

```
ALTER TABLE Student
```

```
DROP COLUMN Address;
```

(v) Insert and display records

```
INSERT INTO Student VALUES ('101', 'Reema', '01-JAN-2005', 'MCA', 2, '9876543210',  
'reema@gmail.com');
```

```
SELECT * FROM Student;
```

(b) PL/SQL Procedure

```
CREATE OR REPLACE PROCEDURE display_student IS
```

```
BEGIN
```

```
    FOR rec IN (SELECT * FROM Student) LOOP
```

```
        DBMS_OUTPUT.PUT_LINE(
```

```
            rec.Reg_no || ' ' || rec.Name || ' ' || rec.Branch
```

```
        );
```

```
    END LOOP;
```

```
END;
```

```
/
```

```
BEGIN
```

```
    display_student;
```

```
END;
```

```
/
```

2. (a) Create a table Employee with the following attributes

(EmpNo (Primary Key), Name, Department, Job_desig,

DOJ, Basic, DA, HRA, Net Salary) and perform the

following :

- (i) Insert data into the table
- (ii) Display all employee details whose
Job_desig=Accountant.
- (iii) Display the names of employee whose Net Salary is
more than 80000.
- (iv) Display the months between the DOJ and till date.
- (b) Write a PL/SQL block to display the net salary of a
particular employee using functions.

(a) SQL Commands

Create Table

```
CREATE TABLE Employee (  
    EmpNo INTEGER PRIMARY KEY,  
    Name VARCHAR(50),  
    Department VARCHAR(30),  
    Job_desig VARCHAR(30),  
    DOJ DATE,  
    Basic NUMBER,  
    DA NUMBER,  
    HRA NUMBER,  
    Net_Salary NUMBER  
);
```

(i) Insert Data

```
INSERT INTO Employee VALUES  
(101, 'Reema', 'Accounts', 'Accountant', '01-JAN-2020', 30000, 5000, 5000, 40000);
```

```
INSERT INTO Employee VALUES  
(102, Leena, 'HR', 'Manager', '15-MAR-2019', 70000, 10000, 10000, 90000);
```

(ii) Employees with Job_desig = Accountant

```
SELECT * FROM Employee  
WHERE Job_desig = 'Accountant';
```

(iii) Names with Net Salary > 80000

```
SELECT Name FROM Employee  
WHERE Net_Salary > 80000;
```

(iv) Months between DOJ and today

```
SELECT Name, MONTHS_BETWEEN(SYSDATE, DOJ) AS Months_Worked  
FROM Employee;
```

(b) PL/SQL Function

```
CREATE OR REPLACE FUNCTION get_salary(e_no NUMBER)
```

```
RETURN NUMBER IS
```

```
    sal NUMBER;
```

```
BEGIN
```

```
    SELECT Net_Salary INTO sal
```

```
    FROM Employee
```

```
    WHERE EmpNo = e_no;
```

```
    RETURN sal;
```

```
END;
```

```
/
```

```
DECLARE
```

```
    result NUMBER;
```

```
BEGIN
```

```
    result := get_salary(101);
```

```
    DBMS_OUTPUT.PUT_LINE('Net Salary: ' || result);
```

```
END;
```

/

3. Create an “Orders” table with the following attributes (order_id, customer_id, order_amount, order_date)

(a) Using numeric functions of SQL, perform the following:

(i) Calculate the total order amount for each customer

(ii) Calculate the average order amount

(iii) Calculate the total number of orders

(iv) Find the average order amount per customer, rounded to the nearest whole number

(b) Using date functions of SQL, perform the following:

(i) Selecting orders placed on a specific date

(ii) Selecting orders placed in a specific month

(iii) Selecting orders placed on the last day of the month

(iv) Selecting orders where the order_date is within the last 30 days

Create Table

```
CREATE TABLE Orders (  
    order_id INTEGER,  
    customer_id INTEGER,  
    order_amount NUMBER,  
    order_date DATE  
);
```

(a) Numeric Functions

(i) Total order amount for each customer

```
SELECT customer_id, SUM(order_amount) AS Total_Amount  
FROM Orders  
GROUP BY customer_id;
```

(ii) Average order amount

```
SELECT AVG(order_amount) AS Avg_Amount  
FROM Orders;
```

(iii) Total number of orders

```
SELECT COUNT(*) AS Total_Orders  
FROM Orders;
```

(iv) Average per customer (rounded)

```
SELECT customer_id, ROUND(AVG(order_amount)) AS Avg_Amount  
FROM Orders  
GROUP BY customer_id;
```

(b) Date Functions

(i) Orders on a specific date

```
SELECT * FROM Orders  
WHERE order_date = '01-JAN-2024';
```

(ii) Orders in a specific month

```
SELECT * FROM Orders  
WHERE EXTRACT(MONTH FROM order_date) = 1;
```

(iii) Orders on last day of month

```
SELECT * FROM Orders  
WHERE order_date = LAST_DAY(order_date);
```

(iv) Orders in last 30 days

```
SELECT * FROM Orders  
WHERE order_date >= SYSDATE - 30;
```

-
4. Create a table Student_details with the following fields
(Stud_id (Primary Key), Student name Mark1, Mark2, Mark3,
Total, Percentage, Grade) and perform the following :
- (a) Implement a check constraint for Grade ('A', 'B',
'C', 'D', 'E', 'U')
 - (b) Create a view that will keep track of the Student id and
name for the students who have passed. (Grade - A, 'B', 'C',
'D', 'E' — Pass and Grade U — Fail)
 - (c) Create a database trigger so that the total and percentage
of specified marks is automatically inserted whenever a
record is inserted.

Create Table with CHECK constraint

```
CREATE TABLE Student_details (  
    Stud_id INTEGER PRIMARY KEY,  
    Student_name VARCHAR(50),  
    Mark1 NUMBER,  
    Mark2 NUMBER,  
    Mark3 NUMBER,  
    Total NUMBER,  
    Percentage NUMBER,  
    Grade CHAR(1) CHECK (Grade IN ('A','B','C','D','E','U'))  
);
```

(b) Create View (Passed Students)

```
CREATE VIEW Passed_Students AS  
SELECT Stud_id, Student_name
```

```
FROM Student_details
WHERE Grade IN ('A','B','C','D','E');
```

(c) Trigger for Total & Percentage

```
CREATE OR REPLACE TRIGGER calc_marks
BEFORE INSERT ON Student_details
FOR EACH ROW
BEGIN
    :NEW.Total := :NEW.Mark1 + :NEW.Mark2 + :NEW.Mark3;
    :NEW.Percentage := (:NEW.Total / 3);
END;
/
```

5. Consider the following tables with the schema :

Department (DepartmentID, DepartmentName)

Employee (EmployeeID, EmployeeName, DepartmentID, Salary)

Write a subquery in SQL to perform the following:

- (a) To Get Employees of a Specific Department
- (b) To Get Department Name for an Employee
- (c) To Get Total Salary Expenses for Each Department
- (d) To Get Departments with No Employees
- (e) To Find Departments with More Than Three Employees.

(a) Employees of a Specific Department

```
SELECT EmployeeName
FROM Employee
```

```
WHERE DepartmentID = (  
    SELECT DepartmentID  
    FROM Department  
    WHERE DepartmentName = 'Accounts'  
);
```

(b) Department Name for an Employee

```
SELECT DepartmentName  
FROM Department  
WHERE DepartmentID = (  
    SELECT DepartmentID  
    FROM Employee  
    WHERE EmployeeName = 'Reema'  
);
```

(c) Total Salary for Each Department

```
SELECT DepartmentID,  
    (SELECT SUM(Salary)  
     FROM Employee E  
     WHERE E.DepartmentID = D.DepartmentID) AS Total_Salary  
FROM Department D;
```

(d) Departments with No Employees

```
SELECT DepartmentName  
FROM Department  
WHERE DepartmentID NOT IN (  
    SELECT DepartmentID FROM Employee  
);
```

(e) Departments with More Than 3 Employees

```
SELECT DepartmentID
FROM Employee
GROUP BY DepartmentID
HAVING COUNT(*) > 3;
```

6. Consider the following tables with the schema :

Department (deptno, deptname, location)

Employee (empno, ename, jobtitle, management, joiningdate, salary, deptno)

(a) Update the employee salary by 10%, whose experience is greater than 10 years.

(b) Delete the employees, who completed 25 years of service.

(c) Display the manager who is having maximum number of employees

(d) Create a view, which contain employee names and their manager names.

(a) Update salary by 10% (experience > 10 years)

```
UPDATE Employee
SET salary = salary * 1.10
WHERE MONTHS_BETWEEN(SYSDATE, joiningdate)/12 > 10;
```

(b) Delete employees with 25 years service

```
DELETE FROM Employee
WHERE MONTHS_BETWEEN(SYSDATE, joiningdate)/12 >= 25;
```

(c) Manager with maximum employees

```
SELECT management
FROM Employee
GROUP BY management
HAVING COUNT(*) = (
    SELECT MAX(COUNT(*))
    FROM Employee
    GROUP BY management
);
```

(d) View: employee names & manager names

```
CREATE VIEW Emp_Manager AS
SELECT E.ename AS Employee_Name,
       M.ename AS Manager_Name
FROM Employee E, Employee M
WHERE E.management = M.empno;
```

7. Create a Library table and execute the following :

- (a) Display Book title, Book type, author name where author = Balagurusamy and book type = Java.
- (b) Display Author name, Book type, year of publication where year above 2000.
- (c) Display Total no. of copies of books subject wise.
- (d) Display all books group by Book Title
- (e) Create a view to locate the books titled with 'SQL'.

Create Table

```
CREATE TABLE Library (  
    Book_Title VARCHAR(50),  
    Book_Type VARCHAR(30),  
    Author_Name VARCHAR(50),  
    Year_Published NUMBER,  
    No_of_Copies NUMBER,  
    Subject VARCHAR(30)  
);
```

(a) Books by Balagurusamy & Java

```
SELECT Book_Title, Book_Type, Author_Name  
FROM Library  
WHERE Author_Name = 'Balagurusamy'  
AND Book_Type = 'Java';
```

(b) Books after year 2000

```
SELECT Author_Name, Book_Type, Year_Published  
FROM Library  
WHERE Year_Published > 2000;
```

(c) Total copies subject-wise

```
SELECT Subject, SUM(No_of_Copies) AS Total_Copies  
FROM Library  
GROUP BY Subject;
```

(d) Display all books grouped by title

```
SELECT Book_Title
```

FROM Library
GROUP BY Book_Title;

(e) View for books titled 'SQL'

CREATE VIEW SQL_Books AS
SELECT *
FROM Library
WHERE Book_Title LIKE '%SQL%';

8. Create the following relational schema

(a) Course (Courseid, Coursename, Coursefee)

Student (Studentid, Studentname, Courseid)

(b) Write SQL queries to implement

(a) Create Tables

CREATE TABLE Course (
 Courseid INTEGER PRIMARY KEY,
 Coursename VARCHAR(50),
 Coursefee NUMBER
);

CREATE TABLE Student (
 Studentid INTEGER PRIMARY KEY,
 Studentname VARCHAR(50),
 Courseid INTEGER,
 FOREIGN KEY (Courseid) REFERENCES Course(Courseid)

(b) Simple SQL Queries

- **Student with course name**

```
SELECT Studentname, Coursename  
FROM Student, Course  
WHERE Student.Courseid = Course.Courseid;
```

- **Students in a course**

```
SELECT Studentname  
FROM Student  
WHERE Courseid = 1;
```

- **Highest course fee**

```
SELECT MAX(Coursefee)  
FROM Course;
```

- **Count students in each course**

```
SELECT Courseid, COUNT(*)  
FROM Student  
GROUP BY Courseid;
```

- **Students without course**

```
SELECT Studentname  
FROM Student  
WHERE Courseid IS NULL;
```

9. Consider the following schema for an order processing database :

Customer (Customerno , Custname, City)

Cust_Order (Orderno, Orderdate Date, Ord_Amt, Customerno)

(a) Implement a check constraint to check City in (Chennai,
Bangalore, Delhi)

(b) Create a synonym for Cust_Order table

(c) Create a view that will keep track of the details of each
customer and the number of orders placed by each

customer

(d) Write a PL/SQL procedure to display all the customer orders.

(a) CHECK constraint

```
CREATE TABLE Customer (  
    Customerno INT PRIMARY KEY,  
    Custname VARCHAR(50),  
    City VARCHAR(20) CHECK (City IN ('Chennai','Bangalore','Delhi'))  
);
```

Cust_Order

```
CREATE TABLE Cust_Order (  
    Orderno INT,  
    Orderdate DATE,  
    Ord_Amt NUMBER,  
    Customerno INT  
);
```

(b) Synonym

```
CREATE SYNONYM O FOR Cust_Order;
```

(c) View

```
CREATE VIEW Cust_View AS  
SELECT Customerno, COUNT(*) Total_Orders  
FROM Cust_Order  
GROUP BY Customerno;
```

(d) Procedure

```
CREATE OR REPLACE PROCEDURE show_orders IS
BEGIN
    FOR i IN (SELECT * FROM Cust_Order) LOOP
        DBMS_OUTPUT.PUT_LINE(i.Orderno || ' ' || i.Ord_Amt);
    END LOOP;
END;
/
```

10. Consider the following relational schema for Sales database :

Product (Productid, Productname, Price, Stock)

Purchase (Purchaseid, Productid, quantity, supplierName)

(a) Create a primary key on Productid and identify the foreign key

(b) Create a view which displays Product ids and quantity in sales

(c) Create a sequence named Purchaseid_Sequence that gets incremented by 1 and use it for inserting Purchaseid values in Purchase table.

(d) Write a PL/SQL procedure to display all the product details.

(a) Keys

```
CREATE TABLE Product (
    Productid INT PRIMARY KEY,
    Productname VARCHAR(50),
    Price NUMBER,
```

Stock NUMBER

);

CREATE TABLE Purchase (

Purchaseid INT,

Productid INT REFERENCES Product(Productid),

Quantity NUMBER,

SupplierName VARCHAR(50)

);

(b) View

CREATE VIEW V AS

SELECT Productid, Quantity FROM Purchase;

(c) Sequence + Insert

CREATE SEQUENCE S START WITH 1 INCREMENT BY 1;

INSERT INTO Purchase

VALUES (S.NEXTVAL, 101, 5, 'ABC');

(d) Procedure

CREATE OR REPLACE PROCEDURE P IS

BEGIN

FOR i IN (SELECT * FROM Product) LOOP

DBMS_OUTPUT.PUT_LINE(i.Productid||' '||i.Productname);

END LOOP;

END;

/
